RNAP Hack 2026

Guía de GitHub · control de avances del equipo

GitHub guarda el historial del proyecto de cada equipo. Cada checkpoint queda registrado con fecha, hora y autor, y con ese registro se revisa el avance de cada fase. Esta guía usa solo el navegador y Colab, sin terminal.

1. Crear el repositorio

Lo hace el responsable del repositorio el jueves 5, en clase.

1. En github.com, botón **New** (o el signo + arriba a la derecha › New repository).
2. Nombre del repositorio: `rnap-hack-<nombre-del-equipo>`, en minúsculas y sin espacios.
3. Visibilidad: **Public**. Si el equipo prefiere **Private**, el docente debe quedar como colaborador para poder revisarlo.
4. Marcar **Add a README file** y elegir la plantilla `.gitignore` de **Python**.
5. Botón **Create repository**.

2. Agregar colaboradores

1. Dentro del repositorio: **Settings › Collaborators › Add people**.
2. Agregar el usuario de cada integrante del equipo y el del docente: _____.
3. Cada invitado acepta la invitación desde su correo o desde github.com/notifications.

3. Estructura del repositorio

Al subir o crear un archivo, GitHub crea las carpetas cuando se escribe una diagonal en el nombre, por ejemplo `notebooks/entrenamiento.ipynb`.

```
rnap-hack-<nombre-del-equipo>/
  README.md          descripción del producto en inglés
  PROGRESS.md        bitácora de checkpoints en inglés
  notebooks/         cuaderno principal (reporte técnico) y cuadernos de cada integrante
  app/               código de la interfaz (tecnología libre)
  requirements.txt   librerías que usa el proyecto
  data/              muestras pequeñas o el enlace a la fuente
```

4. Guardar un cuaderno de Colab en GitHub

1. En Colab: **Archivo › Guardar una copia en GitHub**.
2. La primera vez, Colab pide autorizar el acceso a la cuenta de GitHub. Si el repositorio es privado, marcar **Incluir repositorios privados**.
3. Elegir el repositorio del equipo y la rama `main`.
4. En la ruta del archivo escribir la carpeta y el nombre, por ejemplo `notebooks/entrenamiento.ipynb`.
5. Escribir el mensaje del commit (ver sección 7) y presionar **Aceptar**.

> *Para no sobrescribir el trabajo de otro integrante, cada persona guarda su propio cuaderno con su nombre, por ejemplo `notebooks/ana_baseline.ipynb`. El cuaderno principal `notebooks/entrenamiento.ipynb` lo integra un solo responsable.*

5. Subir o editar archivos desde la web

- **Subir archivos:** en el repositorio, **Add file › Upload files**, arrastrar los archivos, escribir el mensaje del commit y presionar **Commit changes**. El límite por archivo desde la web es de 25 MB.
- **Crear un archivo: Add file › Create new file**, escribir el nombre con su carpeta y el contenido, y presionar **Commit changes**.
- **Editar un archivo:** abrir el archivo, ícono del lápiz, hacer el cambio y presionar **Commit changes**.

6. PROGRESS.md

Cada checkpoint agrega una entrada al inicio del archivo, en inglés, con esta plantilla:

```
## CP2 - 2026-11-05
- What we have done:
- What we haven't done yet:
- Next step:
- Who did what:
```

7. Procedimiento de cada checkpoint

Los checkpoints son los jueves: CP1 y CP2 el 5 de noviembre, CP3 y CP4 el 12 de noviembre, y la versión v1.0 el 19 de noviembre. Entre 10 y 15 minutos antes de que termine cada clase del jueves:

1. Cada integrante guarda en GitHub el trabajo que tiene abierto.
2. El responsable guarda el cuaderno principal y sube el código de la interfaz si cambió.
3. Se agrega la entrada del checkpoint en `PROGRESS.md`.
4. El último commit del checkpoint lleva como mensaje solo su clave: `CP1`, `CP2`, `CP3` o `CP4`.
5. Se revisa en la página principal del repositorio que el commit aparece con la hora correcta.

Fuera de los checkpoints, los mensajes de commit describen el cambio en pocas palabras, por ejemplo `add TF-IDF baseline` o `fix data split`.

8. Versión final v1.0

El jueves 19, antes de las presentaciones, el responsable marca la versión que se presenta:

1. En el repositorio: **Releases › Create a new release** (columna derecha).
2. En **Choose a tag** escribir `v1.0` y elegir **Create new tag**.
3. Título: `v1.0`. Botón **Publish release**.

Lo que se sube después de marcar la versión v1.0 queda fuera de la evaluación de la Fase 3.

9. Reglas

- Cuenta solo lo que se sube durante el horario de clase. El docente revisa la hora de cada push en la vista **Activity** del repositorio, que registra la hora del servidor.
- Cada integrante hace al menos un commit propio en cada fase.
- Un cambio de cientos de líneas entre dos checkpoints sin avance visible en clase se revisa con el equipo.
- Las claves de API nunca se suben al repositorio. En Colab se guardan en **Secretos** (ícono de llave) y en el servicio de publicación, en su sección de variables o secretos.
- Los datos pesados se quedan fuera del repositorio. En `data/` va una muestra pequeña o el enlace a la fuente.

Anexo · Publicar la interfaz

La tecnología de la interfaz y la forma de publicarla son libres. El requisito es que el juez pueda usarla el jueves 19. Estas son opciones gratuitas que funcionan con proyectos de este tamaño:

Opción	Cuándo conviene	Límites a considerar
Hugging Face Spaces (CPU basic)	Interfaces en Python que cargan un modelo de Hugging Face. Acepta Gradio, Streamlit o Docker	Gratuito con CPU. Los modelos grandes tardan en responder
Streamlit Community Cloud	Interfaces hechas en Streamlit, publicadas directo desde el repositorio de GitHub	Gratuito. Memoria de alrededor de 2.7 GB y la app se duerme tras 12 horas sin visitas
Laptop del equipo	Cualquier tecnología que corra en la computadora del equipo	Gratuito. El juez la usa en esa computadora
Enlace temporal desde Colab	Pruebas en clase y respaldo para la demo, por ejemplo `share=True` de Gradio	Gratuito. El enlace caduca y depende de que la sesión de Colab siga abierta
Render (plan gratuito)	Interfaces ligeras que llaman a un modelo por API	Gratuito con 512 MB de RAM. Se duerme tras 15 minutos sin uso y tarda cerca de un minuto en despertar

Antes del jueves 19 se recomienda:

- Probar la interfaz desde una computadora distinta a la del equipo.
- Guardar las claves de API en los secretos del servicio elegido.
- Copiar el enlace o las instrucciones para abrir la interfaz en el `README.md`.
- Tener un respaldo listo, como la laptop del equipo o el enlace temporal desde Colab.